

Software Engineering

Dr. Md. Nasim Adnan

Static Modeling using Class Diagram

Static modeling is the foundation of object-oriented design. A static model shows the conceptual (idea-level) view of a system.

For example, a college library system will have concepts like:

- Book, magazine, library attendant, librarian, student, teacher
- Book issue policy, book return policy, book procurement policy
- A procedure to calculate fines if a book or magazine is returned late
- A procedure to decide if a book or magazine should be placed in the reference section or in the take-home section, and so on.

Similarly, a hospital management system will have concepts like:

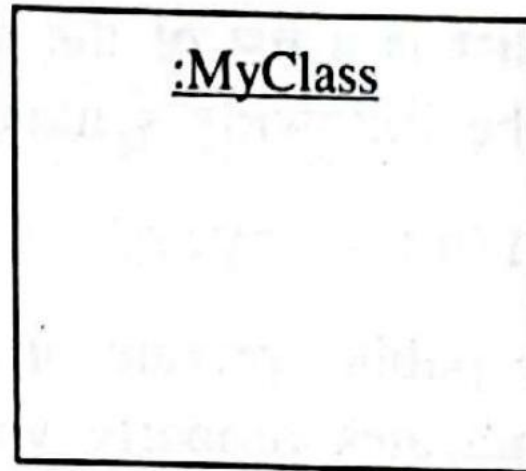
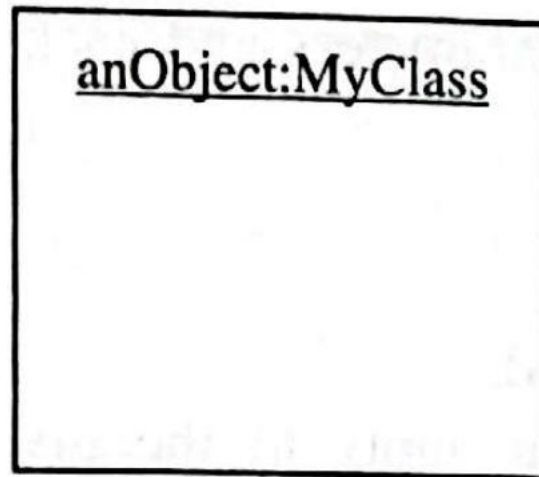
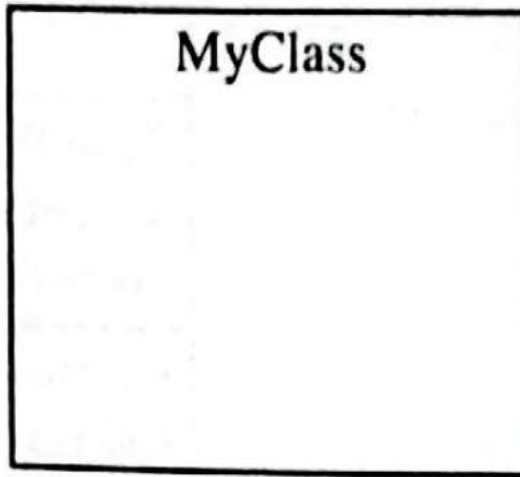
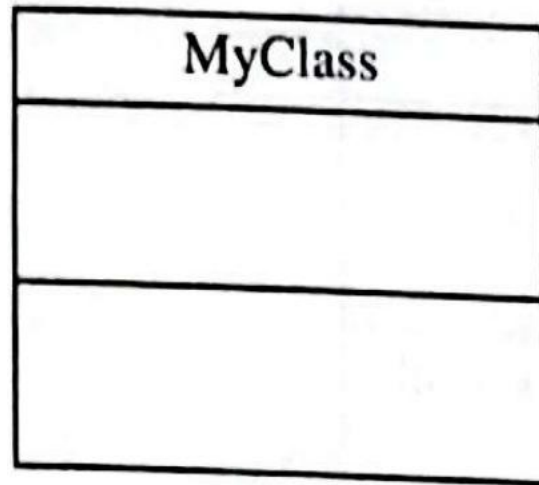
- Rooms and their types, wards, doctors, nurses
- Room allotment policy, medical pharmacy, billing system, etc.

We need to identify the relationships between these concepts so that we can automate both the library management system and the hospital management system. These relationships can be shown visually using class diagrams.

Book Class

- Properties
 - Title
 - Category
 - Author
 - Publisher
 - Catalogue name
 - Accession number
 - Page number

UML: Class Diagram and its Object

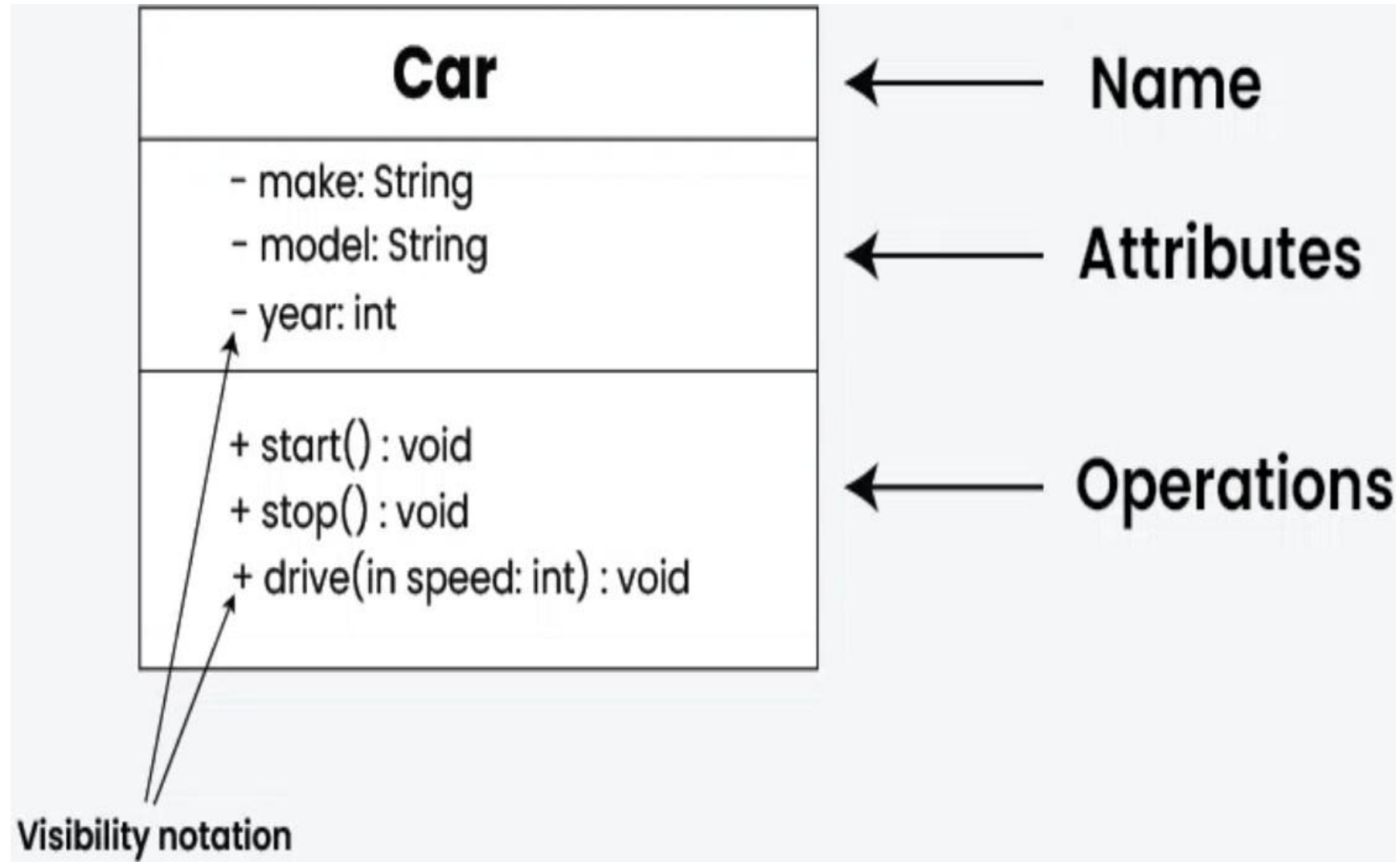


Book

-AccessionNumber : long
-Title : string = ""
-Category : string = ""
-CatalogueNumber : string = ""
-Author : string = ""
-Publisher : string = ""
-PageNumber : int
+IsIssued() : bool

Visibility of Attributes/Properties and Operations/Methods

- Private: -
- Protected: #
- Public: +



Triangle

-base:float
-height:float
-/area:float
-nonOfTriangles:int

+setBase(in b:float):void
+getBase():float
+setHeight(in h:float): void
+getHeight():float
+calculateArea():void
+getArea():float
+getNumberOfTriangles():int

Mapping a Class Diagram to a Programming Language

```
public class Triangle{
    private float base;
    private float height;
    private float area;
    private static int noOfTriangles;

    public void setBase(float b){
    }

    public float getBase(){
    }

    public void setHeight(float h){
    }

    public float getHeight(){
    }

    public void calculateArea(){
    }

    public float getArea(){
    }

    public static int getNumberOfTriangles(){
    }
}
```

Attributes with Default Value

Customer
-custName:String = "Tom" -custID:int = 100
//some operations

The class Customer with attributes having default values.

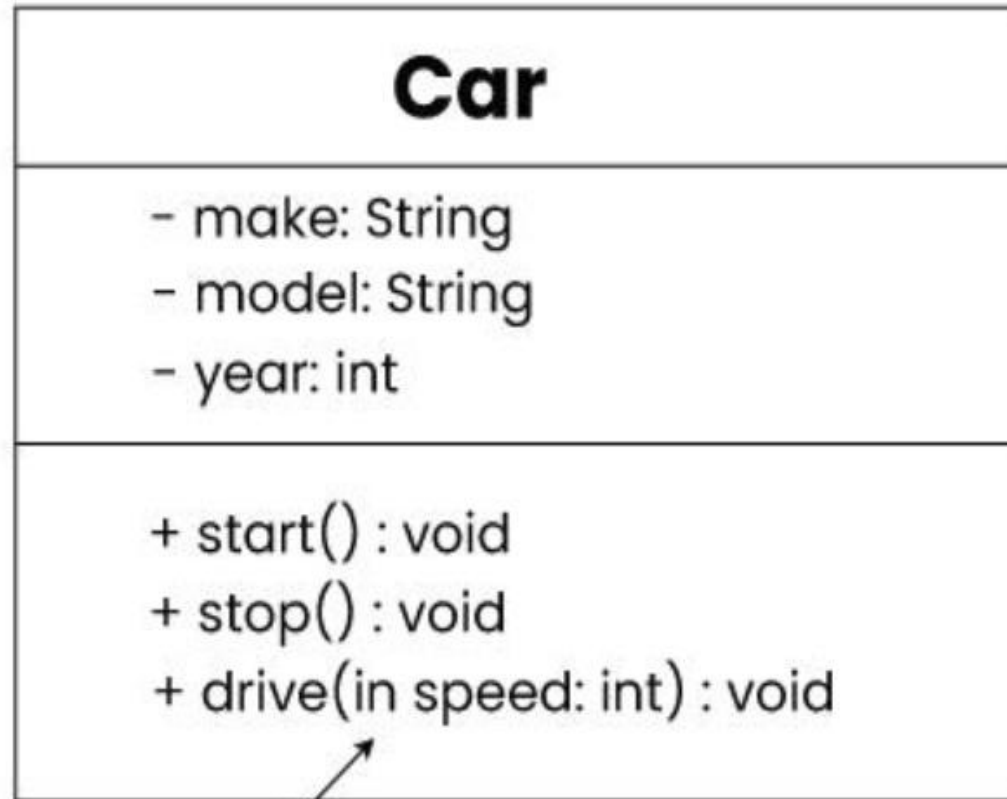
Converting to the Java class, we get

```
public class Customer{  
    private String custName="Tom";  
    private int custId=100;  
}
```

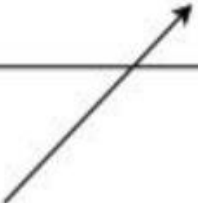
Suppose an object c1 is created. The values for c1's copy of custName and custId will be Tom and 100, respectively.

Parameter Directionality

- In
- Out
- InOut



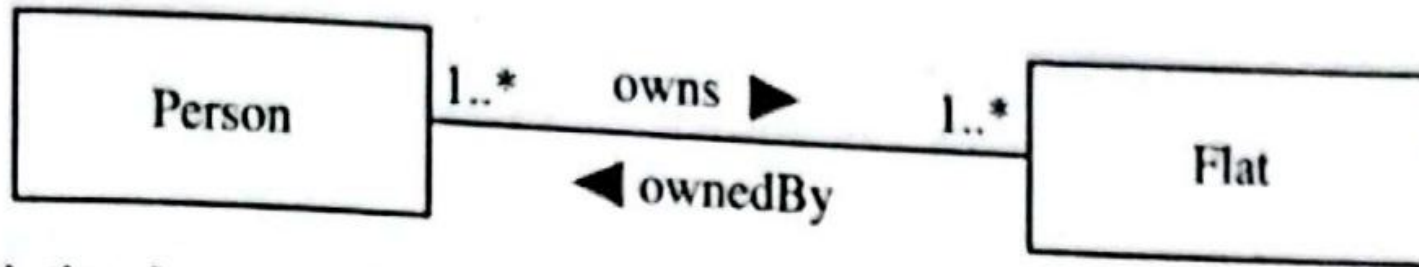
Parameter Directionality



A set of parameters stored together as key-value pairs that a class can use, usually represented as an attribute whose type is a dictionary or map

Relationship between Classes

- Association



Association between Person and Flat. (The association named 'owns' from class Person to class Flat and the association named 'ownedBy' from class Flat to class Person. Multiplicity is also shown).

We can use navigable associations with the navigability shown at the end of an association with the help of an arrow. Thus, if we want to say that “A person owns zero or many flats”, but we do not want to say anything explicitly about a flat being ‘owned by’ person(s), we can draw Figure 3.7.

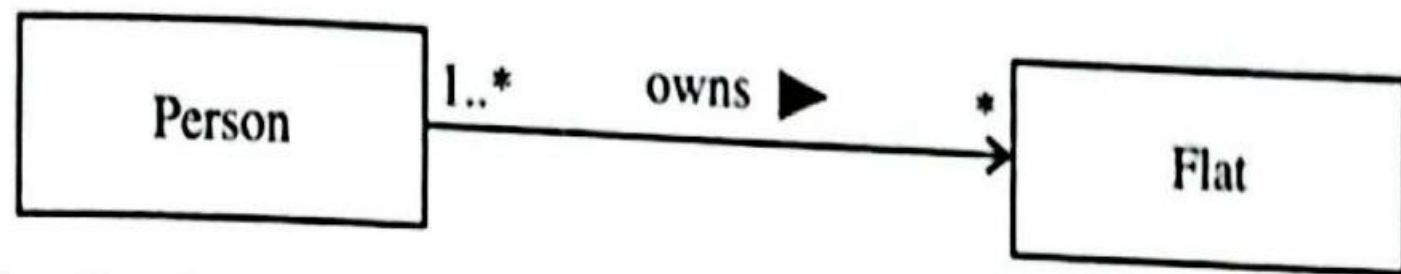
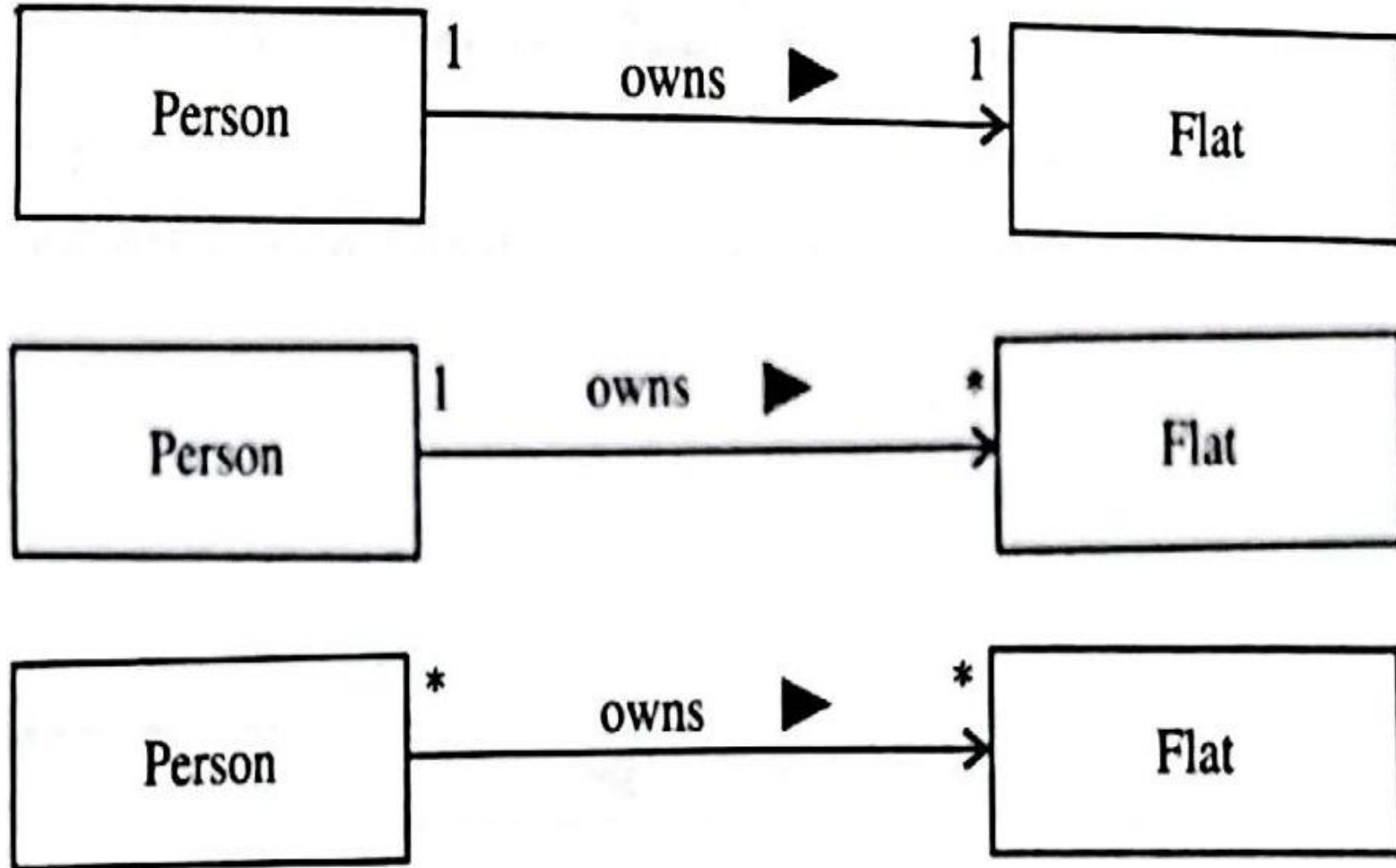
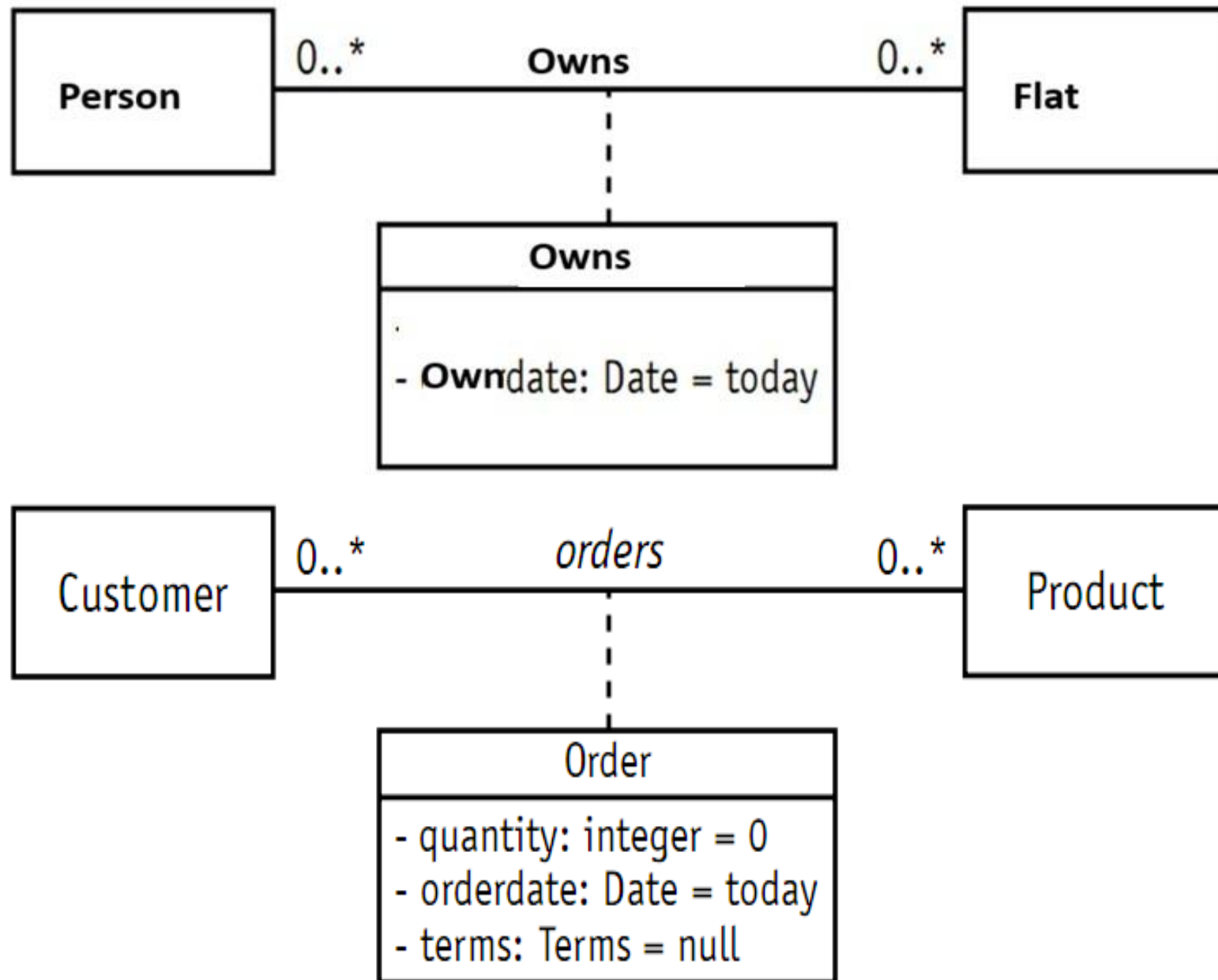


Figure 3.7 Unidirectional association between Person and Flat (The unidirectional association between class Person and class Flat. Multiplicity is also shown).

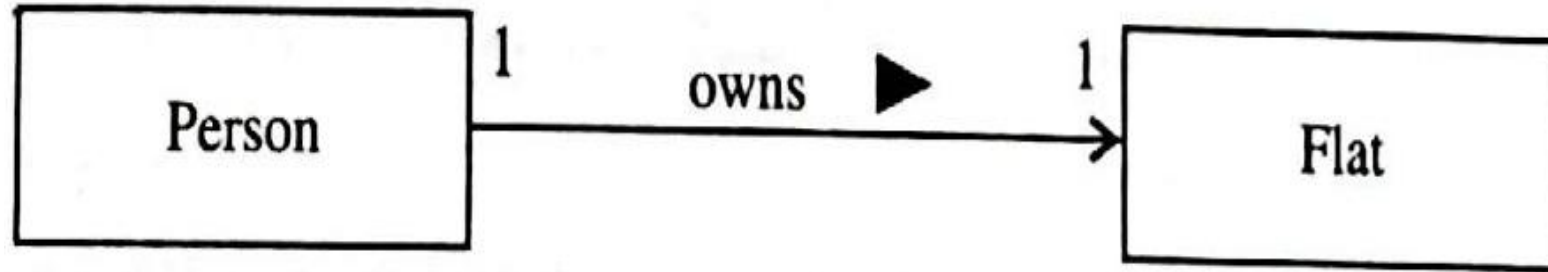
The indicator besides the class Person, in Figure 3.7, i.e., 1..* indicates one or many, whereas * or 0..* indicate zero or many. This indicator is called multiplicity (Fig. 3.7). It can have a fixed value, such as 10 or a range 5..11 or even a series (1, 3, 5, 7, 9). If no multiplicity is specified, then it is 1 by default.

One-to-one, One-to-many and Many-to-many





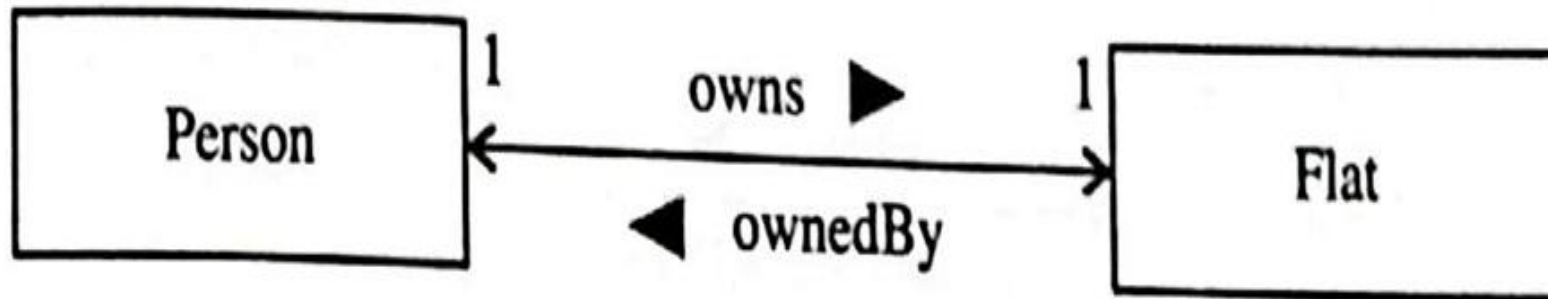
Unidirectional vs. Bidirectional Association



Here Person is Source class and Flat is Target Class

```
public class Person {  
    public Flat owns;  
}
```

```
public class Flat {  
}
```

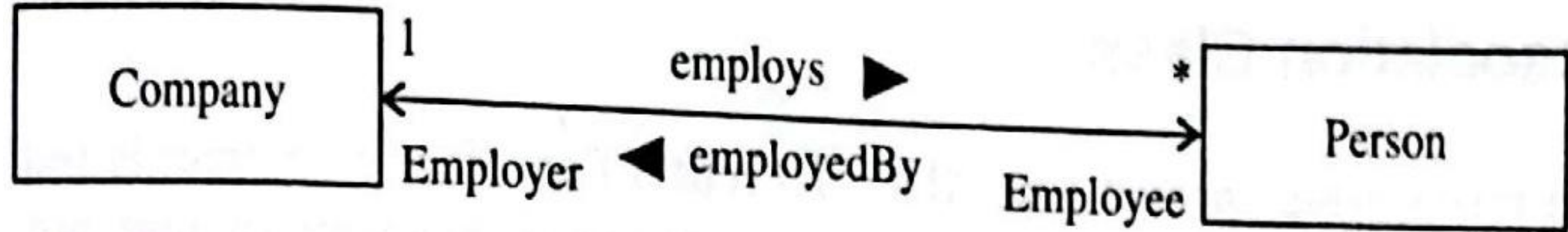


Here owns and ownedBy are Role Names

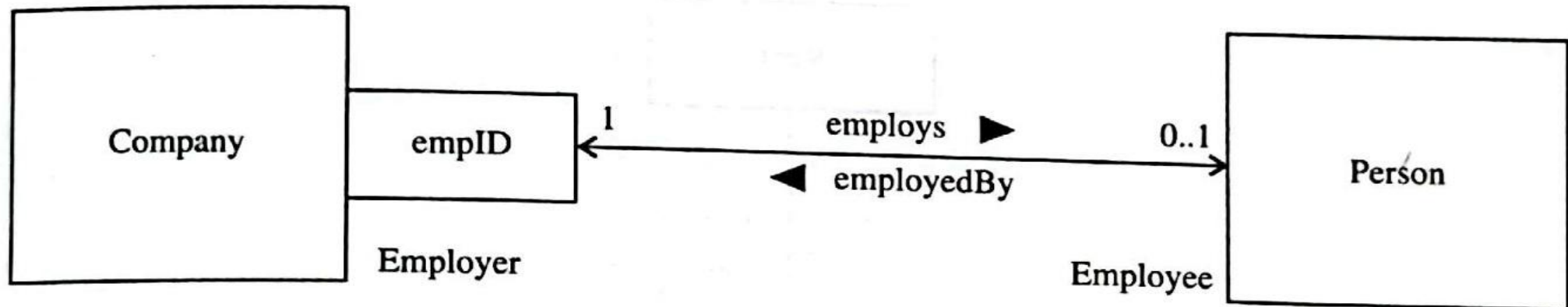
```
public class Person {  
    public Flat owns;  
}
```

```
public class Flat {  
    public Person ownedby;  
}
```

Qualified Association



The qualifier 'empID' distinguishes among the set of objects of the class **Person** at the 'many end' of an association. It indicates how the person is identified by the company, as shown in Figure 3.16.

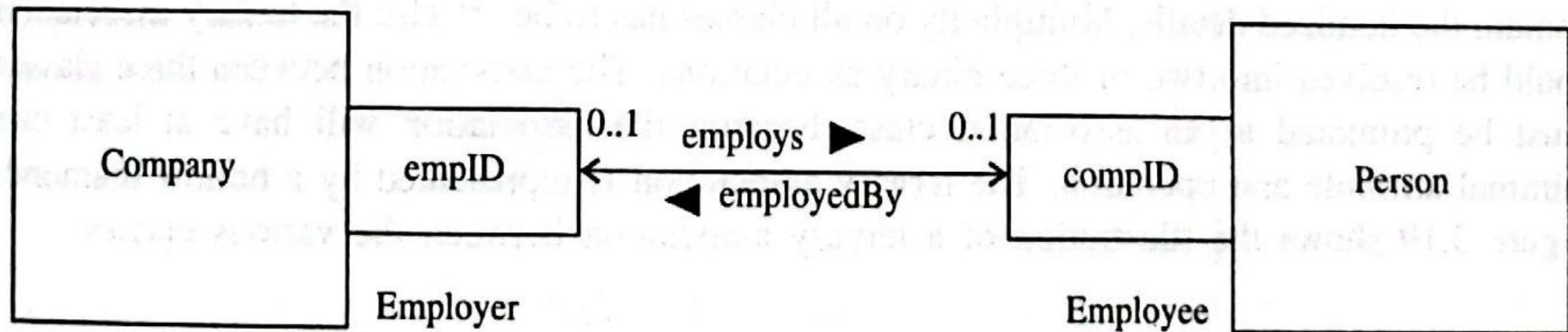


The qualifier is drawn as a small box at the end of an association near the class from which the navigation should be made.

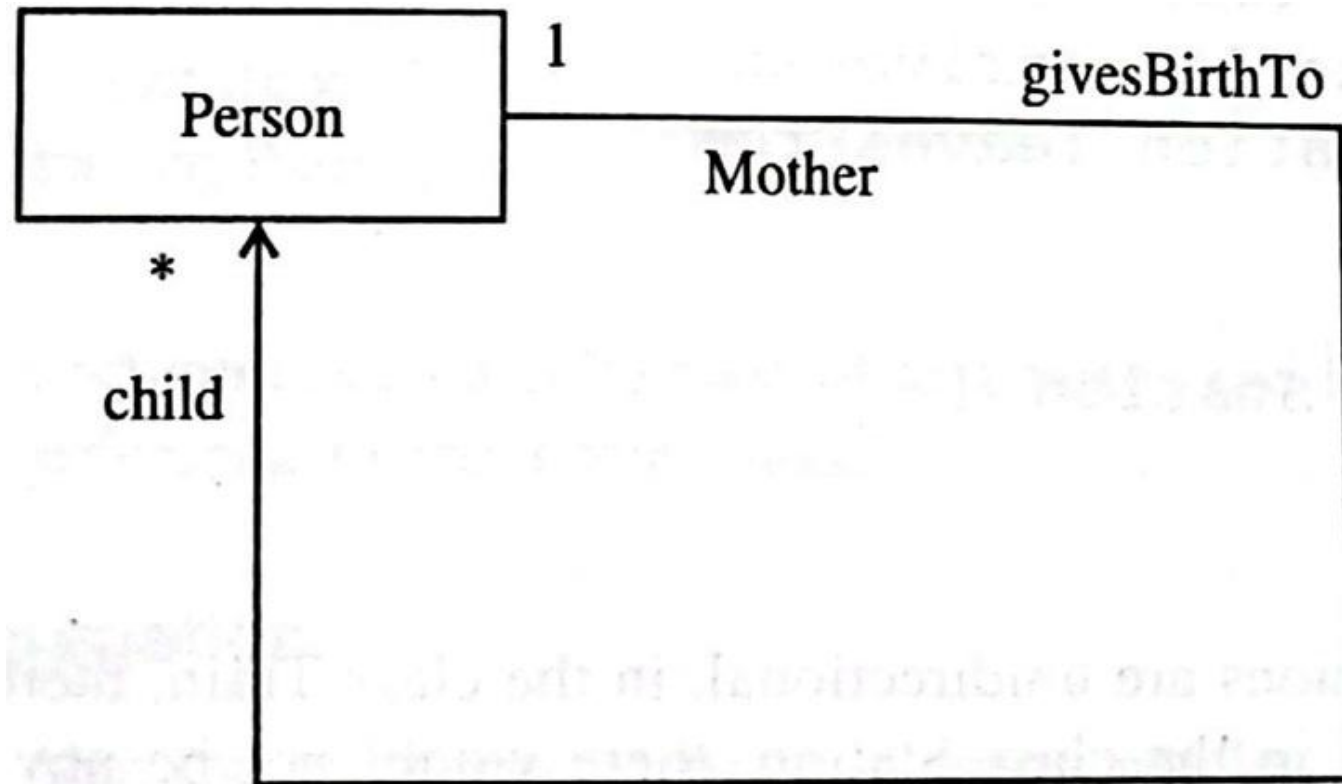
Qualified associations can be used with one to many or many to many associations.

As the navigation is done from Company to Person, i.e., the relationship 'Company employs Person' the many end of the association is 'Person'. So, in order to distinguish each Person, the qualifier empID is used.

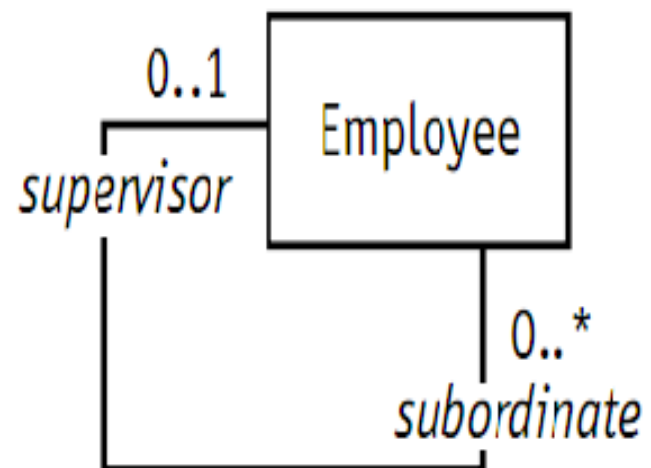
Now a person can be employed by more than one company. So class 'Company' is at the many end of the association. So, the qualifier 'compID' is used to distinguish the Company. This is depicted in Figure 3.17.



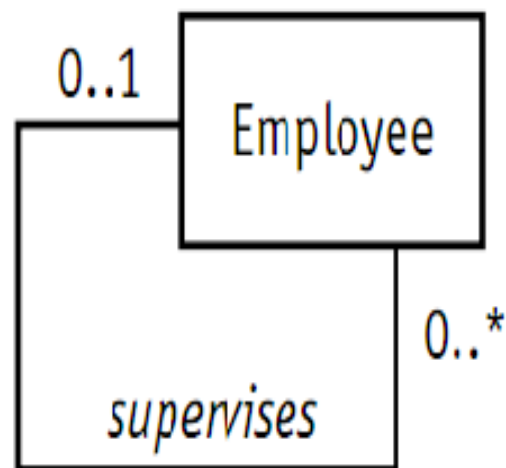
Recursive Association



Using role names

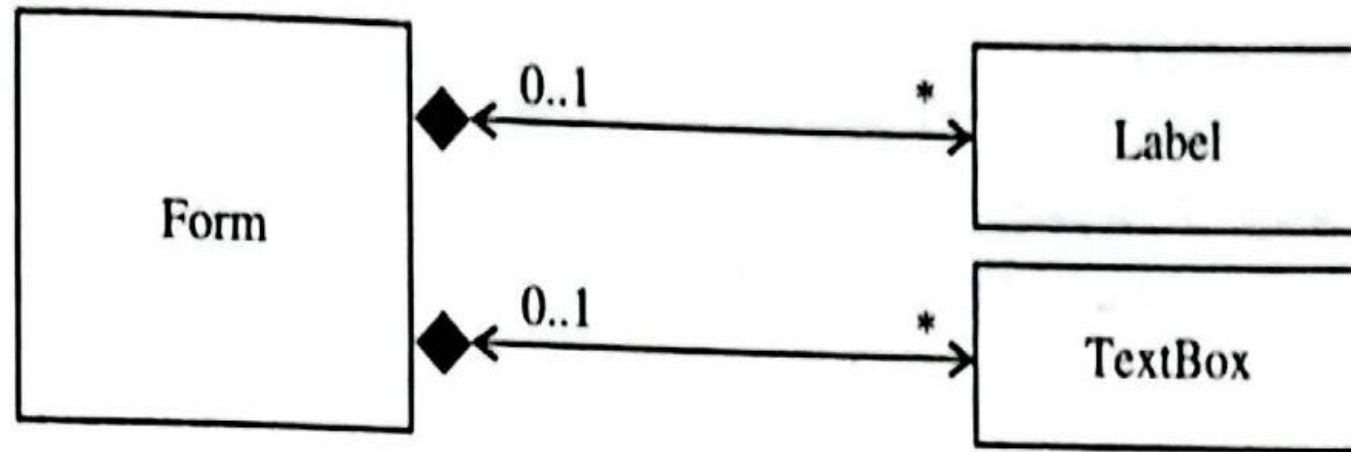


Using an association name



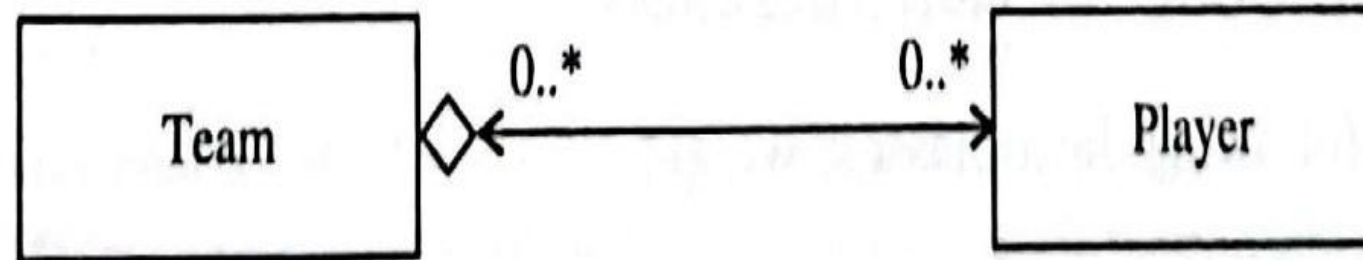
Aggregation: Composite

This aggregation shows that the parts reside within the whole. A Visual Basic Form can contain text boxes, labels, command buttons, etc. If the form is destroyed, then all these parts that reside in the form (whole) will be destroyed. It is indicated by a filled diamond. Multiplicity on the part side can be many but on the whole side it has to be zero or one. Figure 3.22 depicts the composite aggregation.

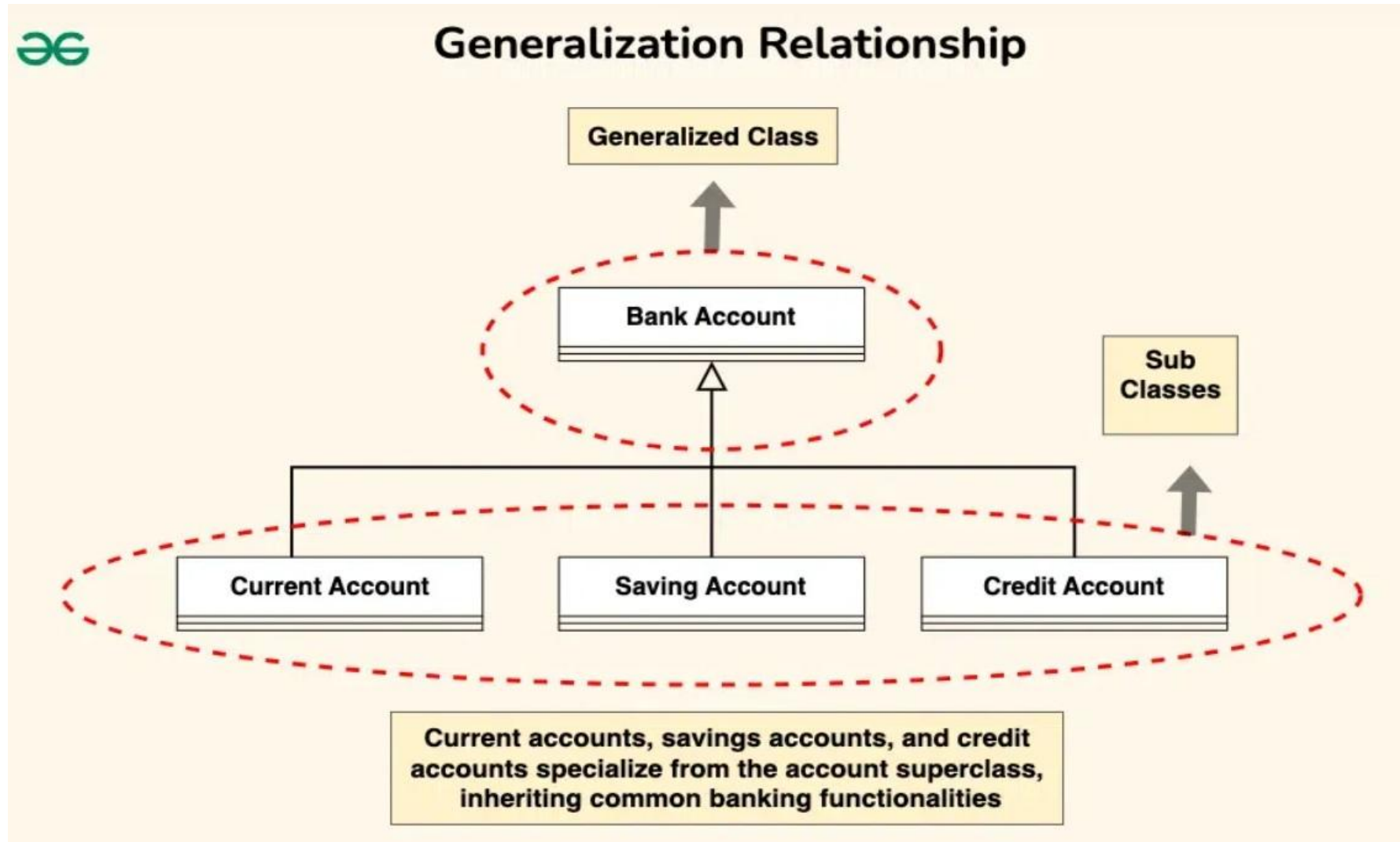


Aggregation: Shared

This aggregation shows that the parts can be parts in many wholes. The multiplicity on the whole side is other than 1. For example, in Figure 3.23, in India, a cricket player can be the member of many teams at the same time: the Maharashtra state team, the Mumbai Ranji team and the national team. Shared aggregation is indicated by a hollow diamond.

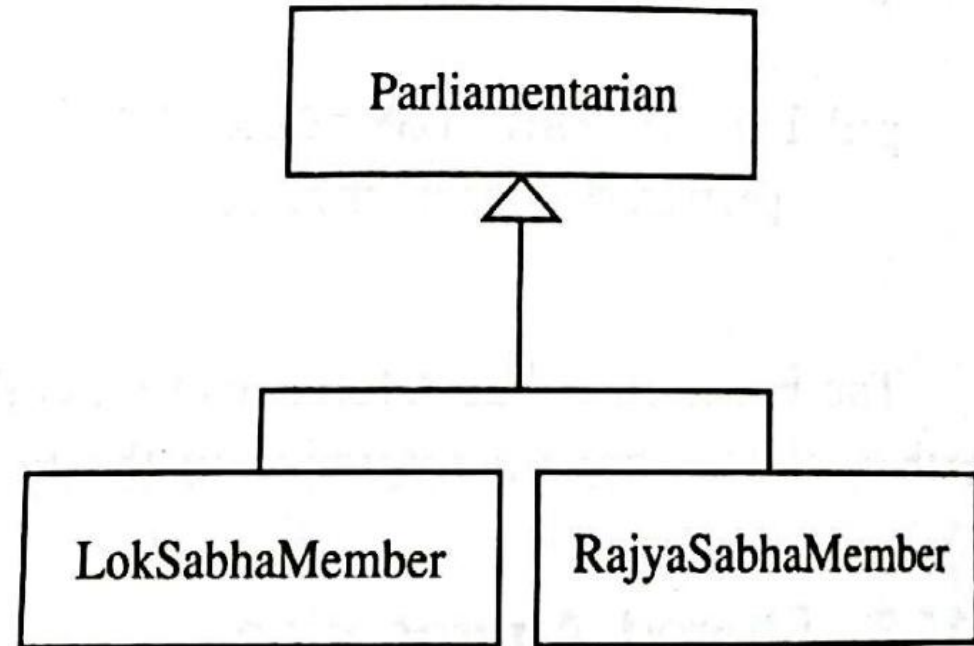
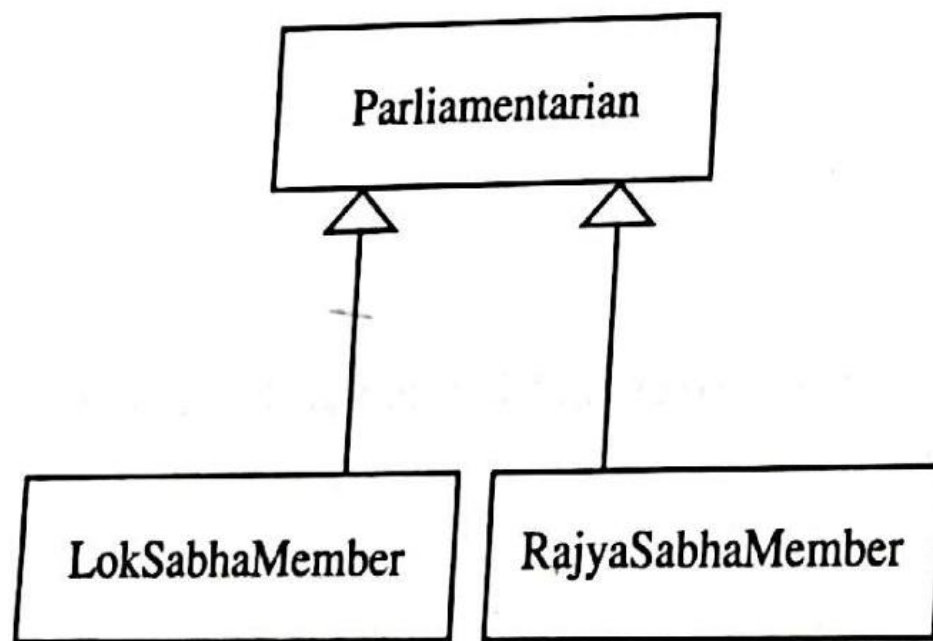


Generalization

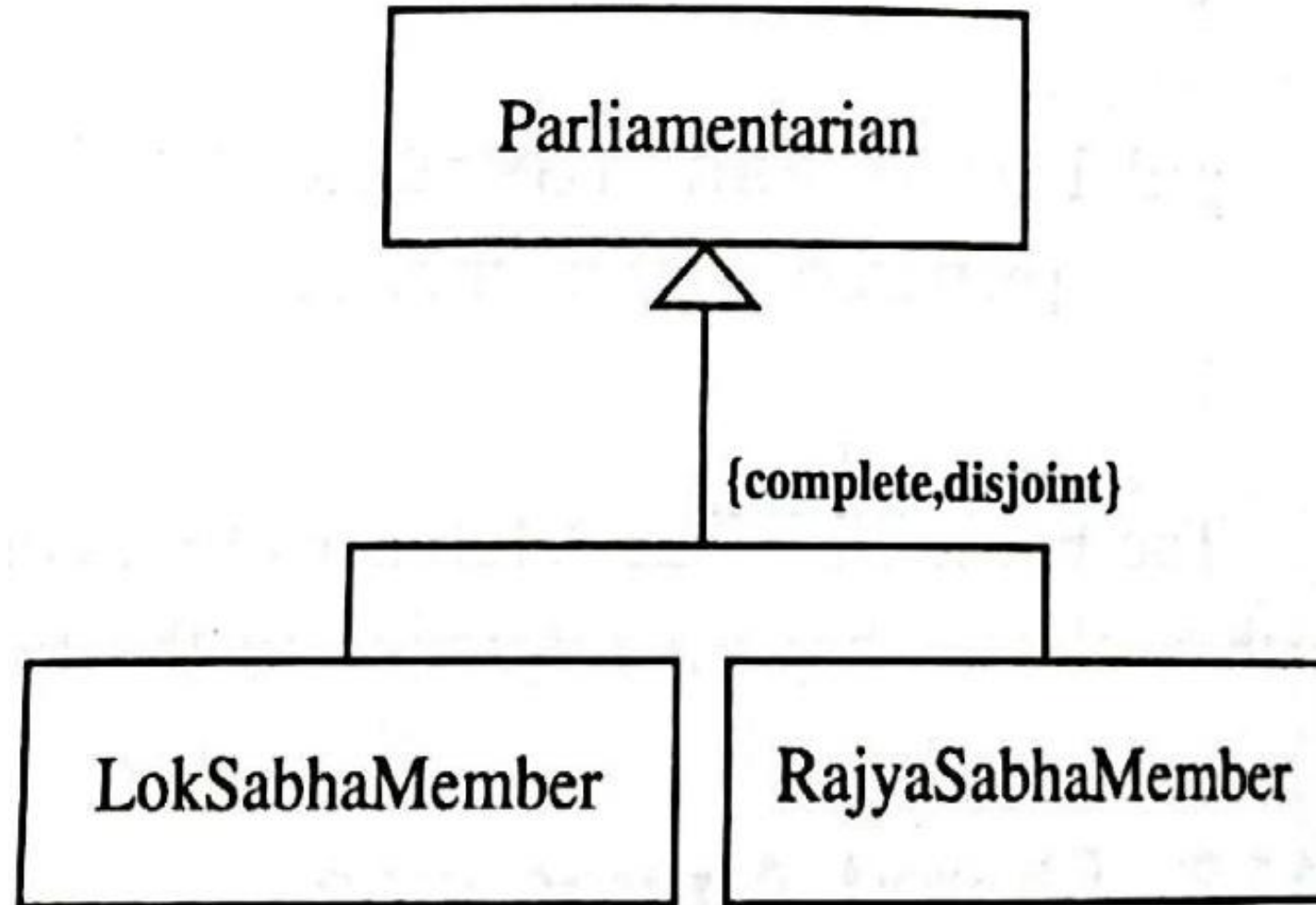


Just as aggregation denotes whole-part relationships, generalization denotes “isa” relationship. For example, in India every member of the House of the People (Lok Sabha) and the Council of States (Rajya Sabha) is a Parliamentarian

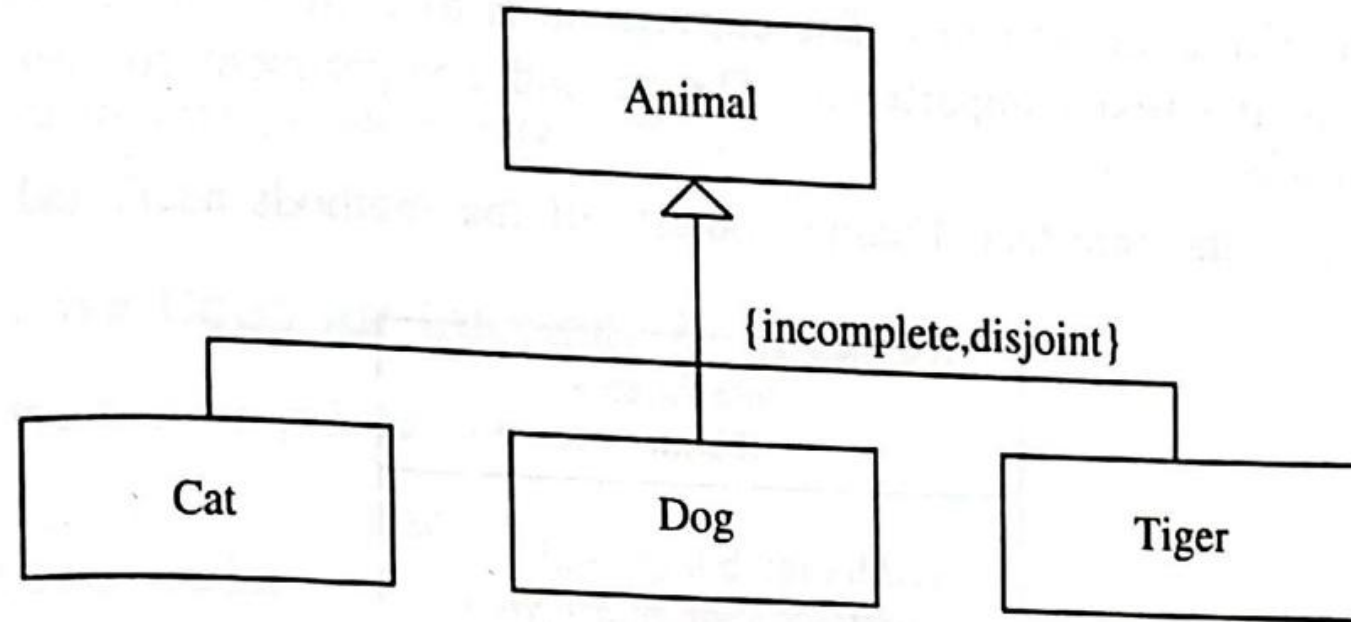
The generalization relationship is denoted by a solid line with a hollow triangle. The head points to the superclass and the tail points to the subclass as depicted in Figure 3.24(a). Alternatively, generalization relationship can be depicted as in Figure 3.24(b).



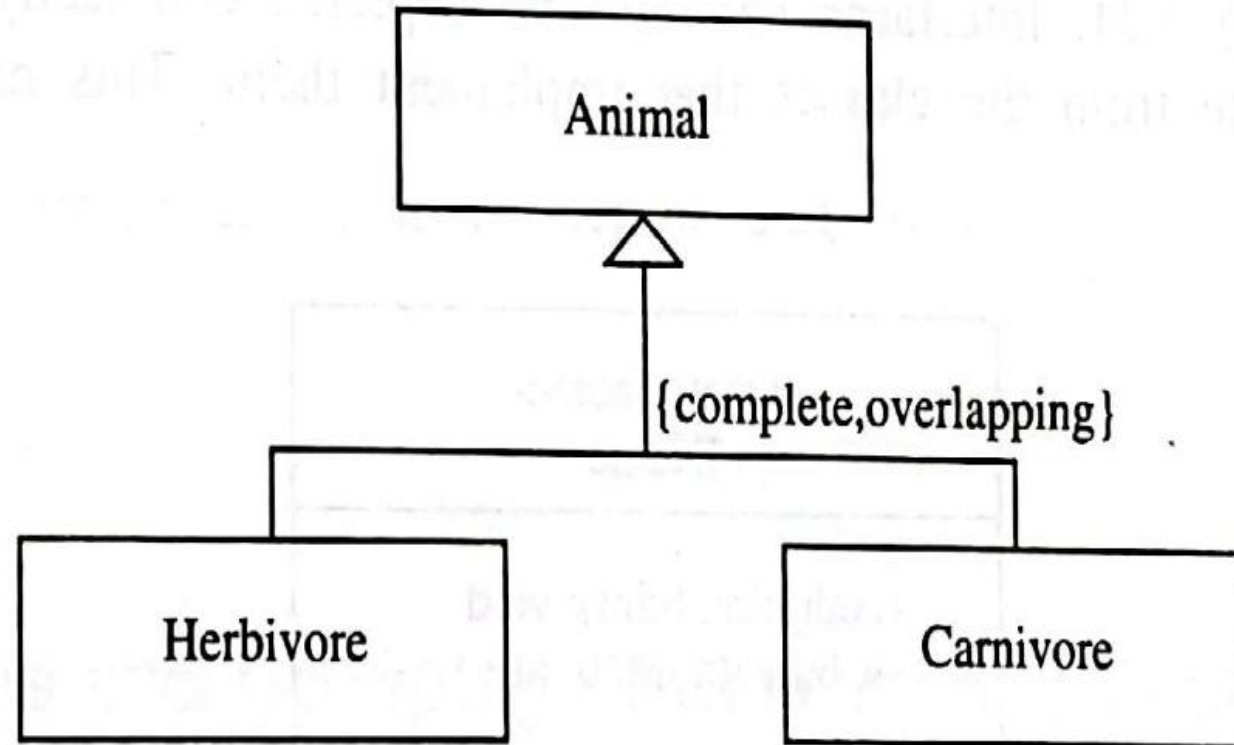
Subclass Partitioning



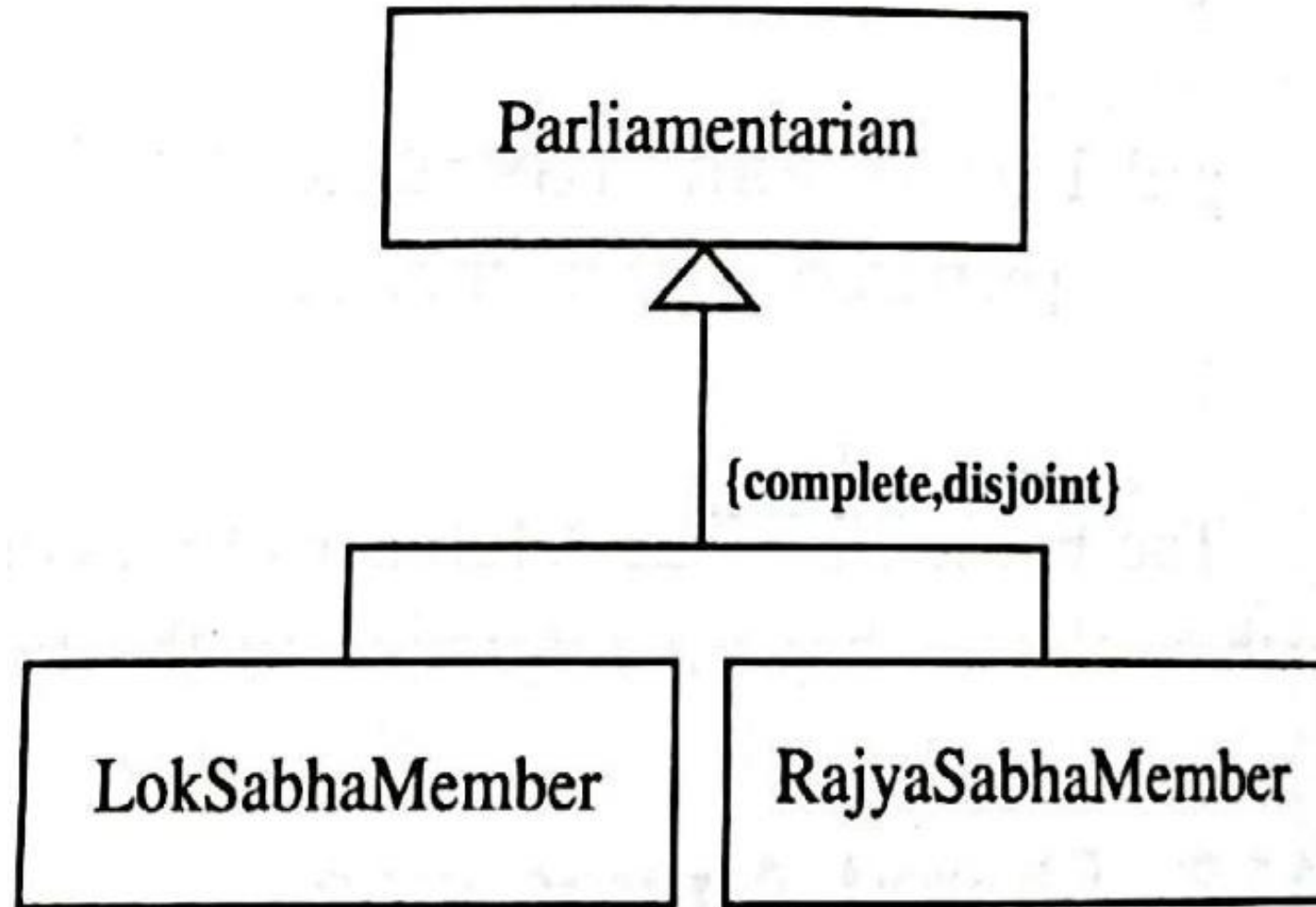
3. Incomplete: It refers to the fact that not all possible subclasses of a class are listed. For example, there may be other subclasses of the superclass Animal such as Cow, Lion, etc., but they are not listed in Figure 3.27. It does not mean that the modeller intends to add them later. It only means that they are not relevant to the model.



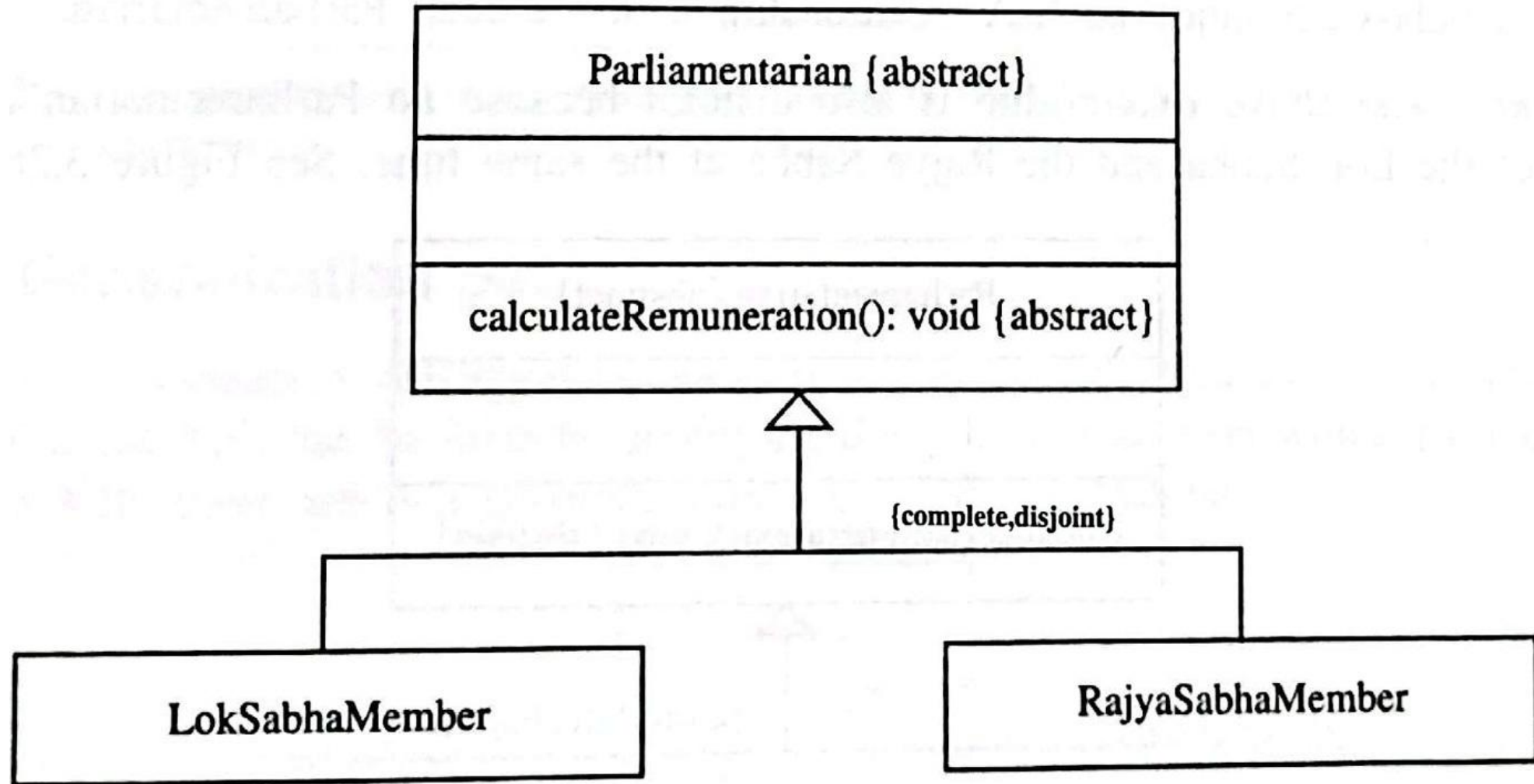
4. Overlapping: The best example is the class Animal partitioned into Herbivore and the class Carnivore. But there can be a class Omnivore (not depicted in Figure 3.28) that has the properties of both. A superclass, partitioned completely, can be made an abstract class.



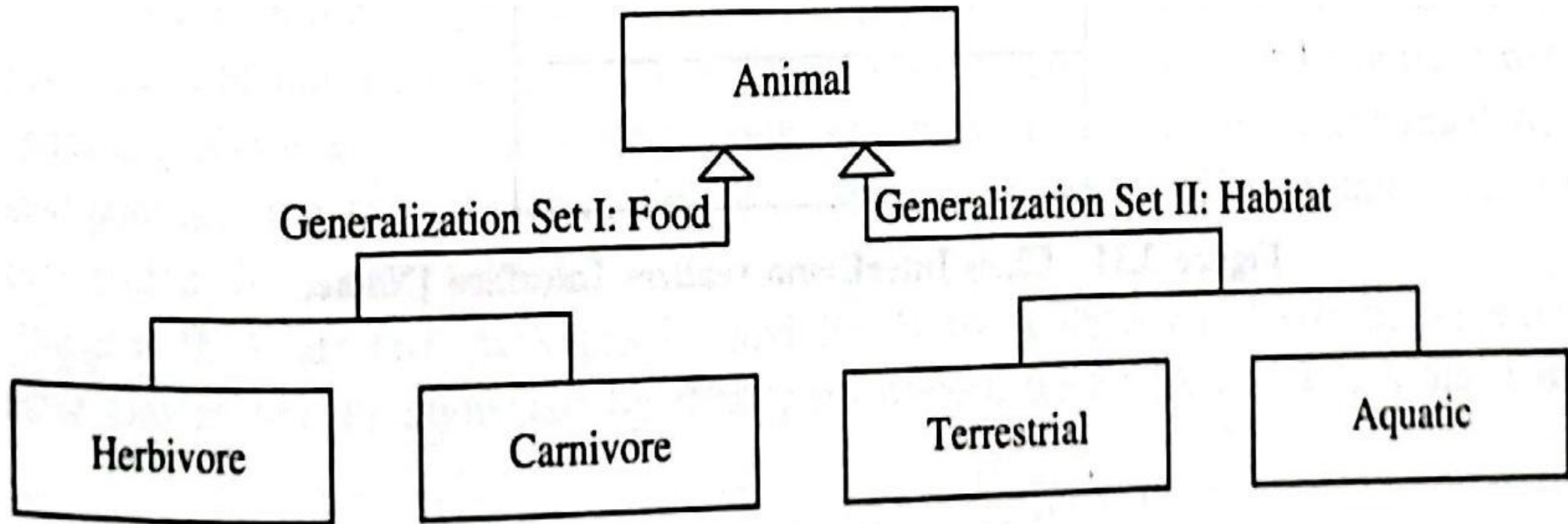
Subclass Partitioning



Abstract Class



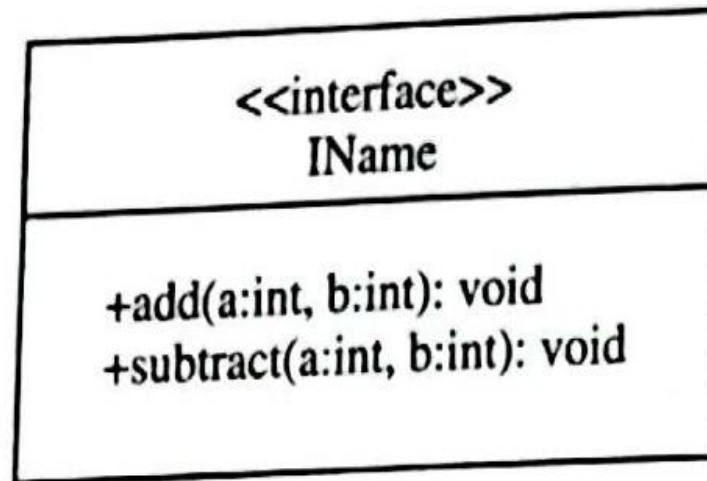
Generalization Set



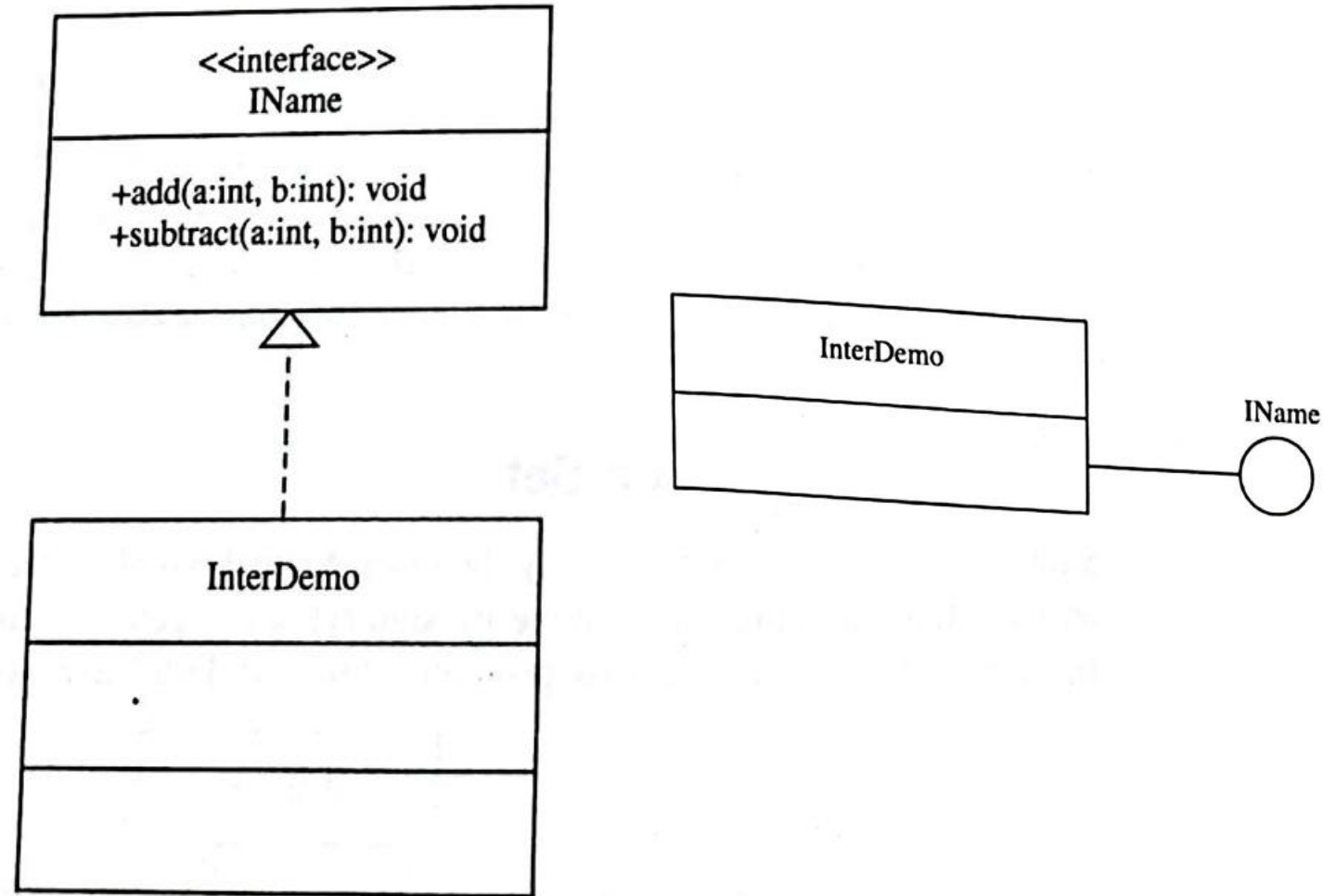
Interfaces

Interfaces are classes that contain only method signatures. They do not specify any implementation, nor are they allowed to have any attributes. An interface is represented by a rectangle having two compartments. The convention is to write <<interface>> and its name on the next line of the first compartment. The second compartment consists of the method names and signatures.

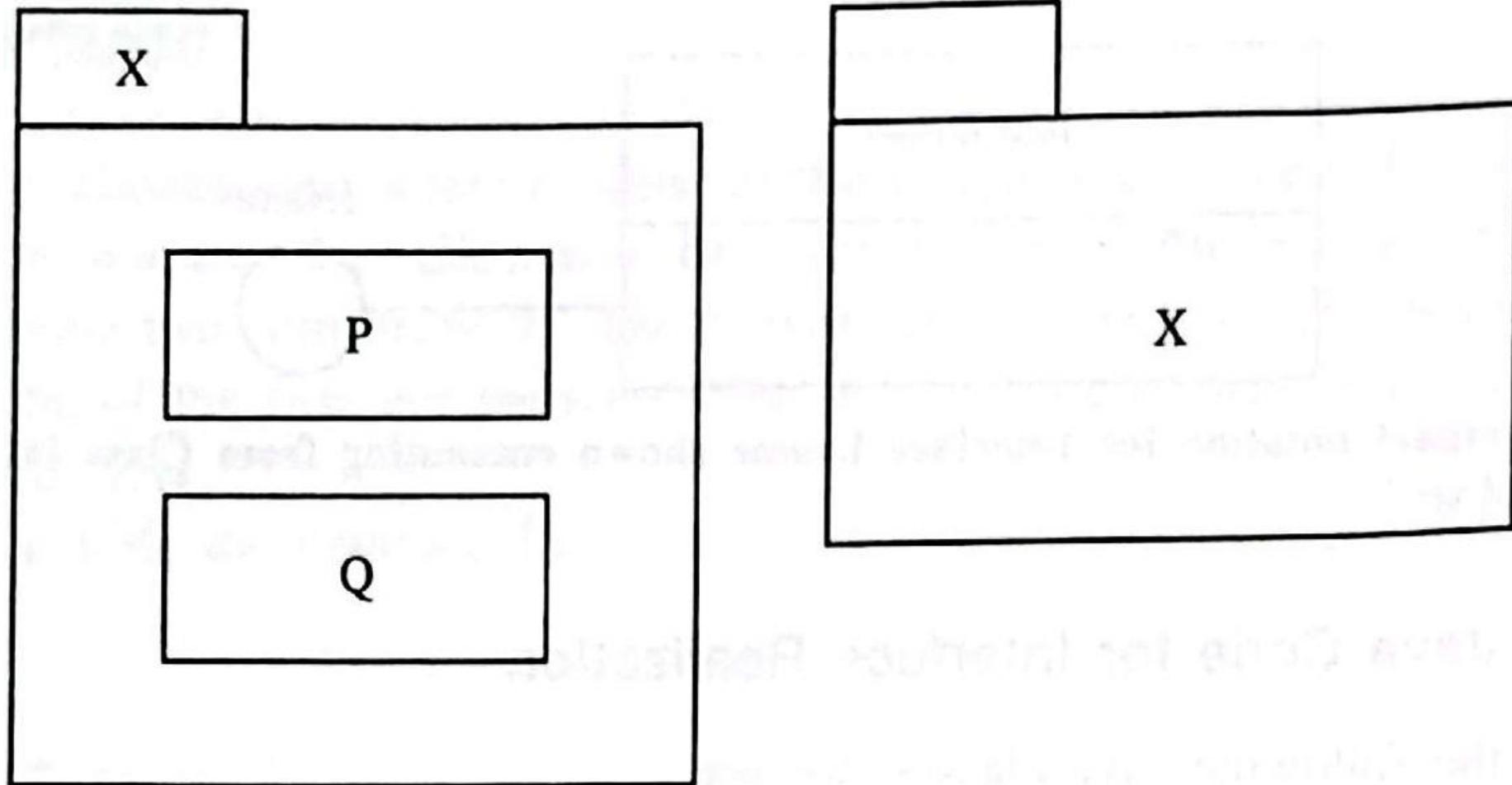
In Figure 3.30, the interface IName consists of the methods add() and subtract().



For an interface to be meaningful, a class should realize it, i.e., a class must implement all methods in the interface. If a class named Interdemo realizes the above interface, it is depicted as in Figure 3.31. Interfaces can also be depicted compactly by a small circle. These circles emanate from the classes that implement them. This notation is shown in Figure 3.32.



Packages and Grouping of Classes into Packages



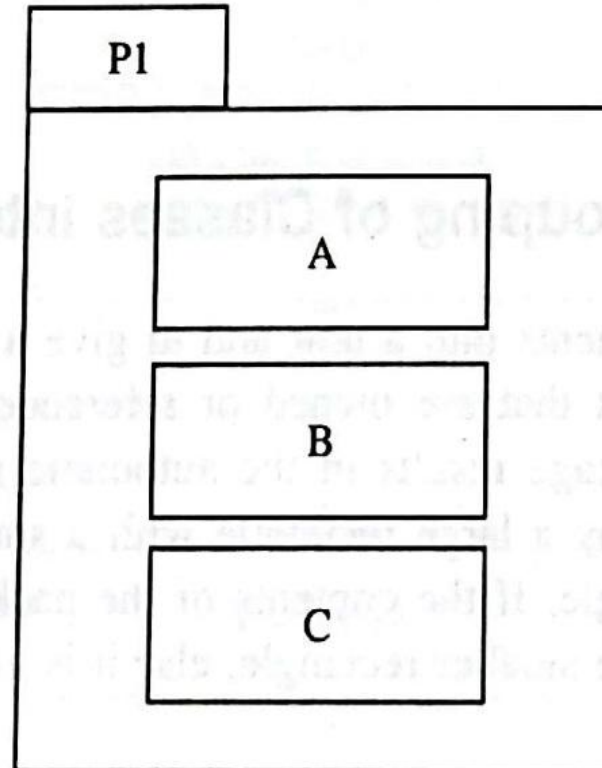
The contents of a package can include other packages, interfaces, components, nodes, etc. Suppose a package has a class A. Another class with the name A cannot be present within the same package. However, it is possible to have an interface or a component named A, within the same package. For the sake of clarity, it is advisable to have different names for different elements within the same package.

Suppose there are two packages P1 and P2. Both of them can have the same class A, and these classes are distinguished by their path-names. So P1::A and P2::A are distinct.

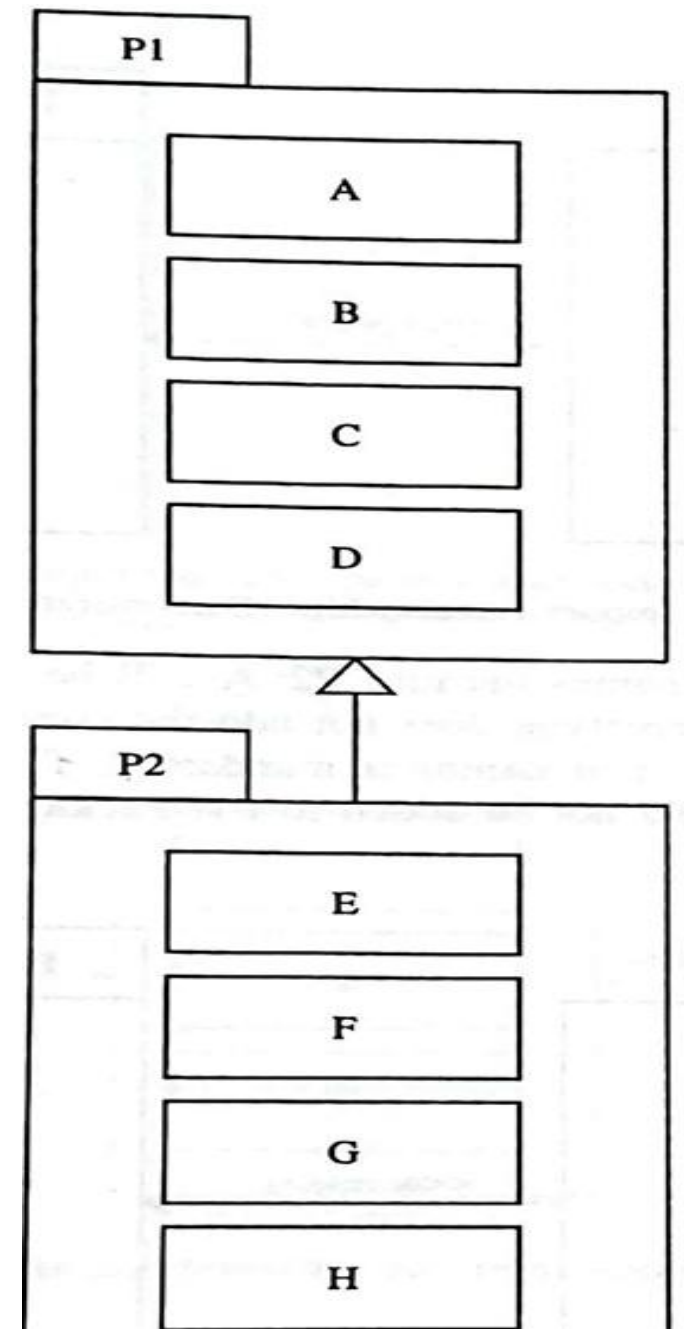
Similar to nested classes are nested packages. Suppose package P3 contains a class 'A'. If P3 is nested within P2 and P2 is nested within P1, then the complete name for class 'A' is P1::P2::P3::A. However, more than three levels of nesting should be avoided as part of good programming practice.

Graphical Display of Owned Elements

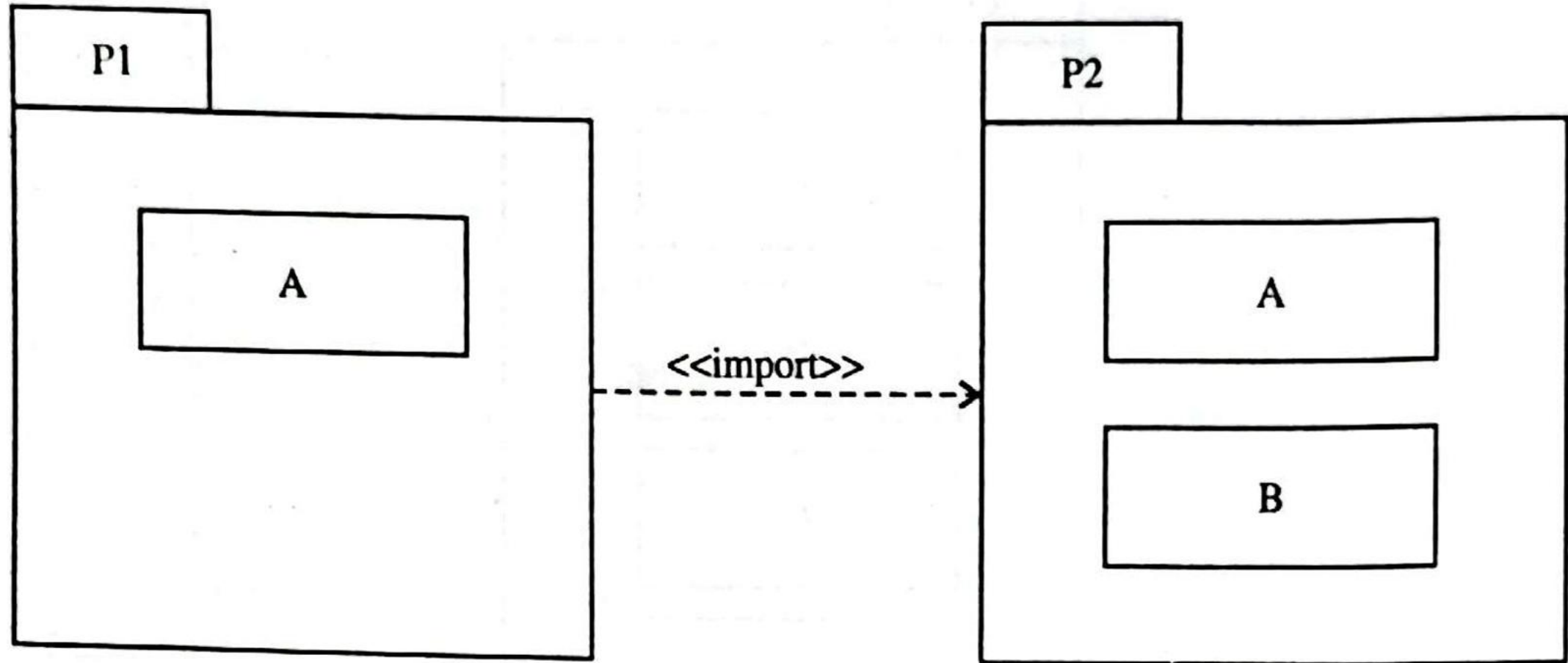
In Figure 3.34, package P1 has three classes A, B and C, which are public, protected and private respectively. A is visible to any package that imports P1; B is visible to those packages which specialize from P1, whereas C is not visible outside package P1.

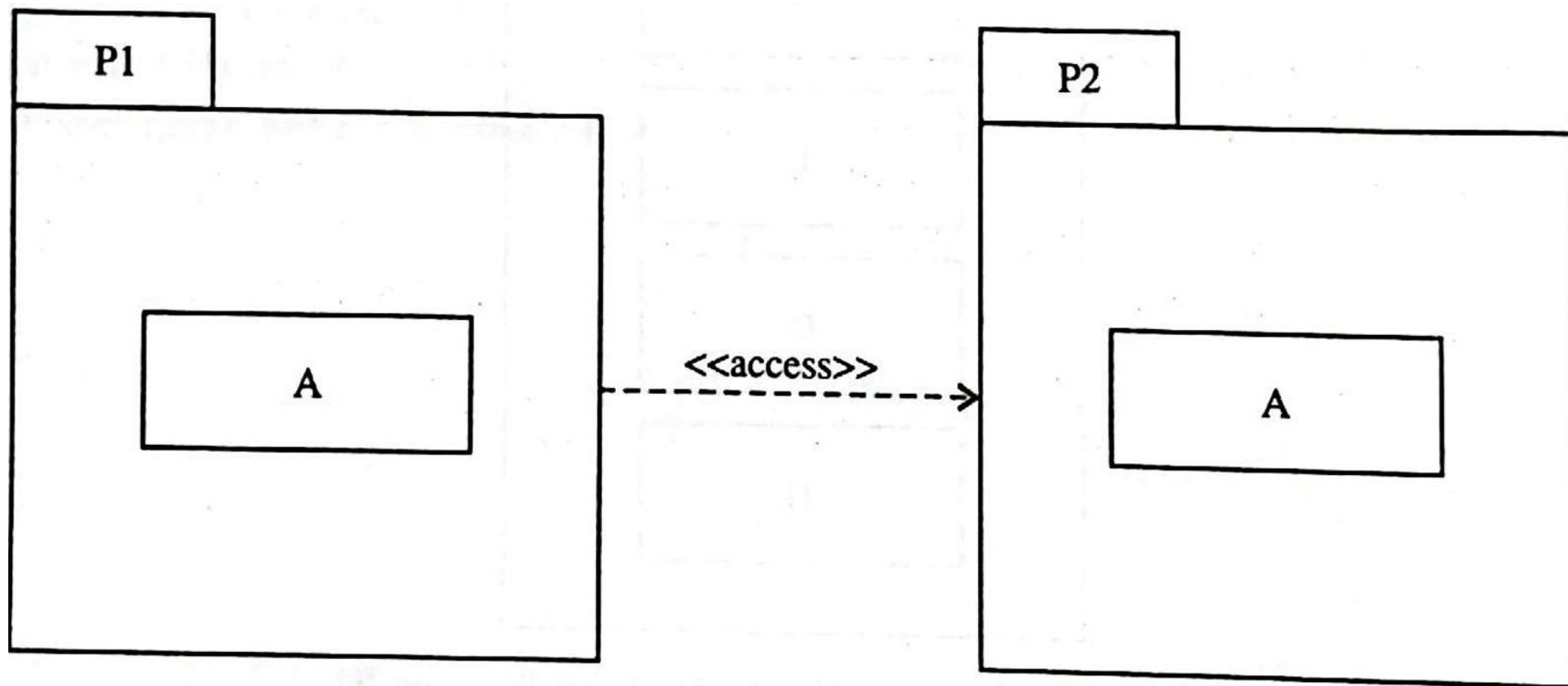


Inherits/Specializes



Import, Access and Export



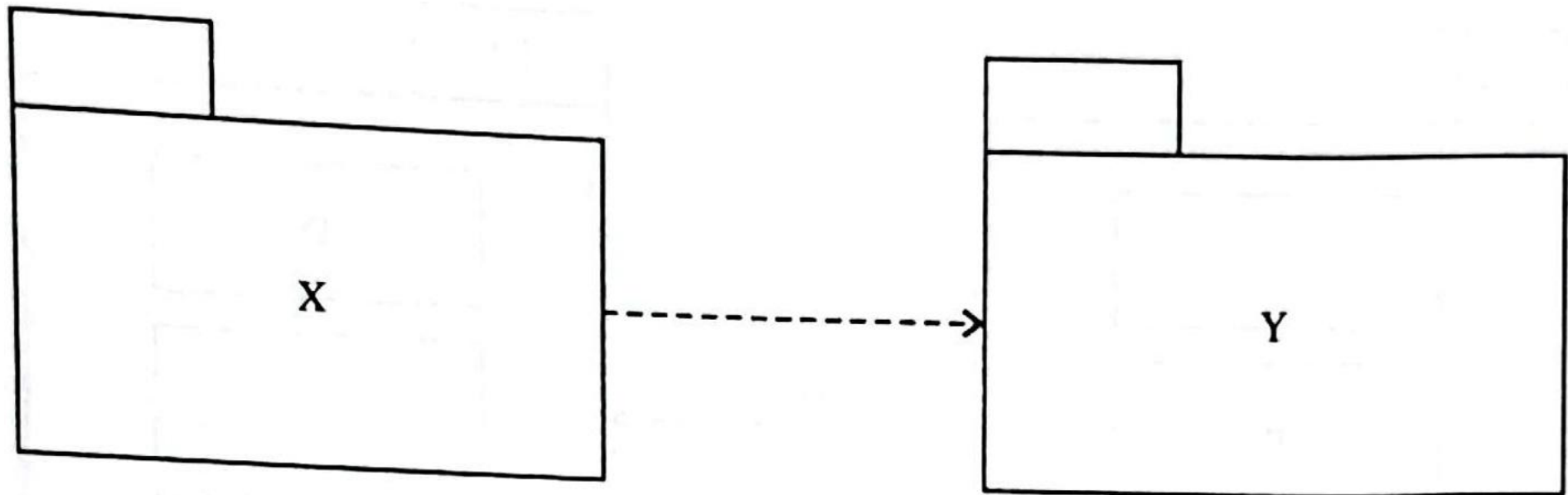


There are three guidelines to consider when grouping classes into a package:

1. Classes belonging to the same inheritance hierarchy.
2. Classes that have an aggregation relationship.
3. Classes that frequently collaborate, as indicated by interaction diagrams.

Dependency

A dependency occurs when one class (Class A) relies on another class (Class B), meaning changes in Class B may require changes in Class A. If a class in Package X depends on a class in Package Y, we say “Package X depends on Package Y.” In system design, it’s important to reduce dependencies between packages to make the system easier to maintain and update. In UML diagrams, a dependency is shown with a dashed arrow (----->) pointing from the dependent element to the element it depends on.



Purpose of Class Diagrams

- The purpose of class diagrams is to visually represent the structure of a system by showing its classes, their attributes, methods, and the relationships between them. They help developers and designers understand how different parts of the system are connected, making it easier to plan, design, and maintain the software. Class diagrams also serve as a blueprint for coding, guide communication among team members, and provide documentation for future reference. This is the only UML that can appropriately depict various aspects of the OOPs concept.
- Proper design and analysis of applications can be faster and efficient.

How to draw Class Diagrams

- Step 1: Identify Classes
- Step 2: List Attributes and Methods
- Step 3: Identify Relationships/Associations
- Step 4: Create Class Boxes
- Step 5: Add Attributes and Methods
- Step 6: Draw Relationships
- Step 7: Label Relationships
- Step 8: Review and Refine
- Step 9: Validate

Identification of Classes and their Attributes

Identification of Relationships/ Associations

